Research Notes on the BA5 2D Commutative Quasigroup Action Key Exchange

Danilo Gligoroski
Norwegian University of Science and Technology (NTNU)
danilog@ntnu.no

21 March 2026

Abstract

These notes document the current BA5 branch-based two-dimensional commutative quasigroup action (CQA) key-exchange prototype implemented in `cqa_2d_kex.py`. The focus is exclusively on the latest BA5 design. The note summarizes the scalar 2-adic CQA background needed to read the implementation, the explicit BA5 balanced-asymmetric five-slot coefficient extractor, the visible two-dimensional swap-family operator shell, the transcript-bound key-derivation layer, the main diagnostic quantities exposed by the implementation, and a revised discussion of shared-secret recovery from two public vectors. Older exploratory variants are intentionally omitted.

Contents

1 Introduction **2**

2 Scalar 2-adic CQA background **3**

2.1 The dyadic ring and odd residues . . . 3

2.2 Public polynomial family . . . 3

2.3 Scalar macro-step ancestry . . . 3

3 The BA5 balanced-asymmetric five-slot extractor **4**

3.1 Public and hidden data . . . 4

3.2 Branch 0 schedule . . . 4

3.3 Branch 1 schedule . . . 5

3.4 Branch iteration depth . . . 5

4 Visible 2D operator shell **5**

4.1 Swap-family matrix . . . 5

4.2 Public key and shared secret . . . 6

5 Transcript-bound session key derivation **6**

6 Diagnostics exposed by the implementation **6**
6.1 Direct one-public-vector recovery test 6
6.2 Swap-basis collapse diagnostics 7
6.3 BA5 public-frame report 7

7 Recovering the shared secret from two public vectors (BA5-specific) **7**

8 Implementation correspondence **9**

9 Remarks on scope **10**

10 How to run the script **10**

11 Acknowledgements and Notes **11**

1 Introduction

Commutative actions generalize Diffie–Hellman-style key exchange beyond groups: the parties apply secret actions to a common public base object, and commutativity guarantees that the two action orders lead to the same shared value. This is the Hidden-Homogeneous-Space (HHS) viewpoint introduced by Couveignes and later exemplified by CSIDH-style class-group actions on supersingular elliptic curves.[Cou06, CLM$^+$18, DF17]

These notes accompany the standalone Python script `cqa_2d_kex.py`. The goal is modest: to record the exact mathematical design implemented by the current BA5 prototype in a compact, self-contained form suitable for publication alongside the code. The present implementation status is a standalone research proof of concept, available as a public GitHub repository.[Gli26]

The underlying design philosophy is to begin with a scalar commutative quasigroup-action-style construction over the dyadic ring

$$R_w = \mathbb{Z}/2^w\mathbb{Z},$$

and then lift this scalar machinery into a visible two-dimensional commutative operator action. In the current implementation, the lift is performed through two scalar coefficient extractors and the public swap-family shell

$$M_K = \lambda_0(K)I + \lambda_1(K)S, \qquad S = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

These notes focus on the BA5 branch design and on the exact implementation currently exposed by the script. Earlier R4-specific material, schedule experiments, and superseded variants are not included here.

2 Scalar 2-adic CQA background

2.1 The dyadic ring and odd residues

The public arithmetic domain is the residue ring

$$R_w = \mathbb{Z}/2^w\mathbb{Z}.$$

Throughout the implementation, parity and 2-adic divisibility are treated as structural design features rather than as incidental implementation details. The odd residues form the unit group

$$R_w^\times = \{x \in R_w : x \equiv 1 \pmod 2\}.$$

The current prototype samples hidden coordinates and the public shell coordinates from the odd residues. This ensures that the hidden tuple lives in the intended odd-key regime and that the visible shell can be studied through 2-adic determinant diagnostics.

2.2 Public polynomial family

The public family map has arity m and is built from slot polynomials

$$P_i(z) \in R_w[z], \qquad i = 1, \ldots, m,$$

combined multiplicatively as

$$f(x_1, \ldots, x_m) = \prod_{i=1}^{m} P_i(x_i) \pmod{2^w}.$$

In the code, each slot polynomial is sampled in a conservative odd-degree regime. The linear coefficient is odd modulo 2^w, while the remaining coefficients are chosen to have controlled 2-adic divisibility. The implementation uses the decomposition

$$P_i(z) = z + 2^s E_i(z), \qquad s = \frac{\texttt{base_w}}{2} - \texttt{delta} + 1,$$

which is the perturbative form used in the BA5 analysis.

A recurring source of intuition comes from polynomial permutations on odd residues modulo powers of two. Markovski–Šunić–Gligoroski give coefficient constraints under which a polynomial induces a permutation of the odd residues modulo 2^n.[MŠG10] In the present work, these results are used only as design intuition rather than as a theorem about the full action: the conservative scalar family samples odd-degree slot-polynomials over R_w in a valuation-controlled manner and keeps the odd part of the dynamics locally permutation-like.[MŠG10]

2.3 Scalar macro-step ancestry

The scalar ancestor of the current design turns a hidden tuple into a unary self-map by cyclically re-inserting a live state variable into a public family map. If

$$K = (k_1, \ldots, k_{m-1}) \in R_w^{m-1},$$

then a scalar macro-step acts on $x \in R_w$ by evaluating the public family repeatedly while rotating the hidden coordinates and reinserting the current state.

This scalar machinery is preserved in the current script through the functions `macro_step` and `act`. The 2D design inherits this ancestry exactly. It does not replace the scalar macro-step machinery; instead, it uses the scalar mechanism twice to extract two scalar coefficients, which are then assembled into a commuting 2×2 operator.

3 The BA5 balanced-asymmetric five-slot extractor

3.1 Public and hidden data

The BA5 extractor uses a public family of arity five. The hidden tuple has length two,

$$K = (k_1, k_2) \in R_w^2,$$

while the public shell coordinates are

$$G = (g_1, g_2) \in R_w^2.$$

Two public scalar seeds

$$u_0, u_1 \in R_w$$

serve as branch inputs, and the visible coefficients are extracted as

$$\lambda_0(K) = T_0(K; u_0), \qquad \lambda_1(K) = T_1(K; u_1).$$

3.2 Branch 0 schedule

The BA5 branch-0 round uses the ordered five-tuple

$$(k_1, k_2, g_1, g_2, x),$$

where x is the live scalar state. One round is defined by

$$y_0 = x,$$

$$\begin{aligned}
y_1 &= P_1(k_1)P_2(k_2)P_3(g_1)P_4(g_2)P_5(y_0),\\
y_2 &= P_1(k_2)P_2(g_1)P_3(g_2)P_4(y_1)P_5(k_1),\\
y_3 &= P_1(g_1)P_2(g_2)P_3(y_2)P_4(k_1)P_5(k_2),\\
y_4 &= P_1(g_2)P_2(y_3)P_3(k_1)P_4(k_2)P_5(g_1),\\
y_5 &= P_1(y_4)P_2(k_1)P_3(k_2)P_4(g_1)P_5(g_2),
\end{aligned}$$

and the one-round branch map is

$$T_0^{(1)}(x) = y_5.$$

3.3 Branch 1 schedule

The BA5 branch-1 round uses the ordered tuple

$$(k_2, k_1, g_2, g_1, x)$$

and changes the feedback-slot path. One round is defined by

$$\begin{aligned}
y'_0 &= x, \\
y'_1 &= P_1(k_2)P_2(k_1)P_3(g_2)P_4(g_1)P_5(y'_0), \\
y'_2 &= P_1(k_1)P_2(g_2)P_3(y'_1)P_4(g_1)P_5(k_2), \\
y'_3 &= P_1(g_2)P_2(g_1)P_3(k_2)P_4(y'_2)P_5(k_1), \\
y'_4 &= P_1(g_1)P_2(y'_3)P_3(k_2)P_4(k_1)P_5(g_2), \\
y'_5 &= P_1(y'_4)P_2(k_2)P_3(k_1)P_4(g_2)P_5(g_1),
\end{aligned}$$

and the one-round branch map is

$$T_1^{(1)}(x) = y'_5.$$

3.4 Branch iteration depth

The implementation applies each branch one-round map repeatedly. If $\rho = \texttt{scalar_rounds}$, then

$$T_0(K; u_0) = \big(T_0^{(1)}\big)^{\rho}(u_0), \qquad T_1(K; u_1) = \big(T_1^{(1)}\big)^{\rho}(u_1).$$

The visible BA5 coefficients are therefore

$$\lambda_0(K) = T_0(K; u_0), \qquad \lambda_1(K) = T_1(K; u_1).$$

4 Visible 2D operator shell

4.1 Swap-family matrix

Once the two scalar coefficients have been extracted, the visible operator attached to the secret tuple K is

$$M_K = \lambda_0(K)I + \lambda_1(K)S, \qquad S = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

Equivalently,

$$M_K = \begin{bmatrix} \lambda_0(K) & \lambda_1(K) \\ \lambda_1(K) & \lambda_0(K) \end{bmatrix}.$$

Because all such matrices are polynomials in the same swap matrix S, the visible operator family is commutative.

4.2 Public key and shared secret

Let $r = \texttt{rounds}$ denote the public vector-round depth. The public key attached to K is the visible orbit point

$$P_K = M_K^r(G).$$

For two users with hidden tuples A and B, the protocol publishes

$$A_{\text{pub}} = M_A^r(G), \qquad B_{\text{pub}} = M_B^r(G),$$

and derives the shared secret by

$$\text{ss}_A = M_A^r(B_{\text{pub}}), \qquad \text{ss}_B = M_B^r(A_{\text{pub}}).$$

Since the visible operators commute,

$$\text{ss}_A = \text{ss}_B = M_A^r M_B^r(G).$$

Remark 4.1. *The implementation exposes this exact semantics: the public key is not $M_K(G)$ but $M_K^r(G)$, and the shared secret is not a one-shot action but the same powered action applied to the other party's public key.*

5 Transcript-bound session key derivation

The raw shared secret is a two-coordinate vector in R_w^2. The implementation serializes the two coordinates in little-endian form, concatenates the public transcript

$$A_{\text{pub}} \parallel B_{\text{pub}},$$

and derives a session key through an HMAC-SHA3-256 HKDF-style layer.

Thus the session key is transcript-bound. The KDF does not authenticate the exchange, but it avoids treating the raw 2-adic vector directly as a final application key.

6 Diagnostics exposed by the implementation

6.1 Direct one-public-vector recovery test

The public key satisfies

$$A_{\text{pub}} = M_A^r(G).$$

At one visible application, the shell matrix acts linearly on the public base vector. The natural determinant controlling direct coefficient recovery is

$$\Delta(G) = g_1^2 - g_2^2.$$

If $\Delta(G)$ is invertible modulo 2^w, then the visible coefficients can be recovered from one public vector by solving the corresponding 2×2 linear system. The current odd-G policy is meant to block this route by forcing $\Delta(G)$ to be even.

6.2 Swap-basis collapse diagnostics

The visible shell has symmetric and antisymmetric channels

$$\mu_+(K) = \lambda_0(K) + \lambda_1(K), \qquad \mu_-(K) = \lambda_0(K) - \lambda_1(K).$$

The implementation tracks several collapse notions derived from these channels:

- whether M_K^r becomes scalar,
- whether one branch dies modulo 2^w,
- whether both branches die,
- whether the powered branch values coincide.

These diagnostics do not prove security; they are sanity checks for visible dimensional collapse.

6.3 BA5 public-frame report

The BA5-specific public frame is organized by the shell quantities

$$D_1 = g_1 u_0 + g_2 u_1, \qquad D_2 = g_1 u_1 + g_2 u_0,$$

together with

$$\Delta_{\mathrm{BA5}} = u_0 u_1 (g_1^2 - g_2^2).$$

The code classifies the visible shell into a small set of regimes according to whether:

(i) the direct one-public-vector route is open,
(ii) the BA5 public-shell Stage-II determinant is invertible,
(iii) the odd-G policy is respected,
(iv) Stage I keeps full 2-adic precision through odd D_1 or D_2.

This classification is not a proof of hardness. It is an implementation-facing summary of which obvious shell-level recovery routes are visibly open or blocked. For the broader passive-view question of whether the transcript $(A_{\mathrm{pub}}, B_{\mathrm{pub}})$ suffices to recover the shared secret, see Section 7.

7 Recovering the shared secret from two public vectors (BA5-specific)

In the actual BA5 protocol, a passive observer sees only the two public orbit vectors

$$A_{\mathrm{pub}} = M_A^r(G), \qquad B_{\mathrm{pub}} = M_B^r(G),$$

where

$$M_K = \lambda_0(K) I + \lambda_1(K) S, \qquad \lambda_0(K) = T_0(K; u_0), \qquad \lambda_1(K) = T_1(K; u_1),$$

and $r = \texttt{rounds}$ is the visible vector-round depth. The public transcript therefore consists of vectors in R_w^2, not matrices. The observer does *not* receive the hidden tuple $K = (k_1, k_2)$, the coefficient pair $(\lambda_0(K), \lambda_1(K))$, the matrix M_K, or the powered matrix M_K^r.

The shared secret is

$$\mathrm{ss} = M_A^r(B_{\mathrm{pub}}) = M_B^r(A_{\mathrm{pub}}) = M_A^r M_B^r(G),$$

so a passive attacker cannot compute the shared secret directly from the transcript unless enough hidden information has first been reconstructed to emulate at least one of the two visible actions.

A convenient way to phrase the public action is

$$\mathcal{A}_K(V) := M_K^r(V), \qquad V \in R_w^2.$$

Then the published values are

$$A_{\text{pub}} = \mathcal{A}_A(G), \qquad B_{\text{pub}} = \mathcal{A}_B(G),$$

and the shared secret is

$$\text{ss} = \mathcal{A}_A(B_{\text{pub}}) = \mathcal{A}_B(A_{\text{pub}}).$$

In this language, a passive attack on the shared secret must reduce to one of the following tasks.

S1. Recover an action-equivalent hidden representation for one party. If an adversary can recover any hidden tuple A' or B' such that

$$\mathcal{A}_{A'}(V) = \mathcal{A}_A(V) \qquad \text{or} \qquad \mathcal{A}_{B'}(V) = \mathcal{A}_B(V)$$

on the relevant public inputs, then the shared secret follows immediately by applying the recovered action to the opposite public vector. Thus shared-secret recovery reduces to action recovery for at least one participant.

S2. Recover the visible coefficient data for one or both parties. Since the BA5 shell is determined by the coefficient pair $(\lambda_0(K), \lambda_1(K))$, one may try to recover the coefficient data directly from

$$A_{\text{pub}} = M_A^r(G), \qquad B_{\text{pub}} = M_B^r(G).$$

In the current odd-G policy, the direct one-public-vector inversion route based on

$$\Delta(G) = g_1^2 - g_2^2$$

is intentionally blocked, because $\Delta(G)$ is even and therefore noninvertible modulo 2^w. The BA5 public-frame determinant

$$\Delta_{\text{BA5}} = u_0 u_1 (g_1^2 - g_2^2)$$

is monitored for the same reason: if it were invertible, the public shell would expose a much more direct Stage-II recovery route.

S3. Jointly recover both hidden branch states from the two public vectors. One may try to use the pair of equations

$$A_{\text{pub}} = M_A^r(G), \qquad B_{\text{pub}} = M_B^r(G)$$

to reconstruct the hidden coefficient state of both parties at once. In the BA5 setting this means recovering enough information about the two hidden tuples to emulate the branch extractors

$$T_0(K; u_0), \qquad T_1(K; u_1)$$

at the chosen branch-depth parameter $\rho =$ `scalar_rounds`. This is a mathematically coherent attack goal, especially in toy-width or low-entropy models, but no explicit efficient method is presently known for the practical-width odd-G regime.

S4. Exploit visible image structure or future 2-adic algebraic attacks. If future work were to show that the BA5 public image occupies a significantly smaller or highly structured subset of R_w^2, then transcript-level precomputation, distinguishing, lifting, or lattice-style attacks might help reduce shared-secret recovery to a more tractable problem. At present, however, the strongest current evidence does not provide such an attack in the practical-width odd-G regime.

The cryptanalytic meaning of this section is therefore narrower than a full security claim. The current data show that the public interface is the orbit map

$$K \longmapsto P_K = M_K^r(G),$$

not the matrix map

$$K \longmapsto M_K \qquad \text{or} \qquad K \longmapsto M_K^r.$$

Hence a passive adversary must begin from two visible vectors rather than from a fully exposed operator. This is exactly why the distinction between vectors and matrices matters in the BA5 design: the honest parties can apply their own hidden operator to the other public vector, but an external observer is forced into an inverse problem for the public orbit map.

Accordingly, the present project does not claim that recovering the shared secret from two public vectors is impossible. The more modest working hypothesis is the following: in the current BA5 design, shared-secret recovery from A_{pub} and B_{pub} is not known to be easier than recovering an action-equivalent hidden representation for at least one of the two parties. Turning that heuristic into a precise hardness statement remains open.

Proposition 7.1 (Transcript-level recovery reduces to public-action inversion)**.** *In the current BA5 public interface, the transcript consists only of the two vectors*

$$A_{\text{pub}} = \mathcal{A}_A(G), \qquad B_{\text{pub}} = \mathcal{A}_B(G),$$

while the internal coefficient data and the operators M_A, M_B remain hidden. Consequently, any passive method that recovers the shared secret from the transcript must in effect reconstruct enough hidden information to emulate at least one of the two public actions on the opposite public vector.

Remark 7.2. *This proposition is intentionally only a structural summary. It does not rule out future algebraic, statistical, or 2-adic attacks. It only records that, in the present BA5 design, the public transcript is an orbit-level interface rather than an operator-level interface.*

8 Implementation correspondence

For convenience, the main mathematical objects correspond to the following functions and classes in `cqa_2d_kex.py`:

Mathematical object	Implementation name
public scalar-family parameters	`Params`, `setup`
slot polynomial family	`PolyOddDegree`, `Family`
scalar macro-step machinery	`macro_step`, `act`
BA5 parameters	`Params2DBA5`
BA5 branch-0 round	`ba5_branch0_round`
BA5 branch-1 round	`ba5_branch1_round`
iterated BA5 branch maps	`scalar_action_T0_ba5`, `scalar_action_T1_ba5`
visible coefficient extraction	`derive_lambdas_ba5`
visible shell matrix	`operator_matrix`
public key	`pubkey_2d`
shared secret	`shared_secret_2d`
session key derivation	`kdf_from_shared_2d`
collapse diagnostics	`degeneracy_report`
public-frame diagnostics	`public_frame_report_BA5`
small regression test	`self_test_ba5_defaults`

9 Remarks on scope

These notes document a research prototype. They are not intended to claim that the current BA5 design is secure. The role of the note is narrower: to state clearly what the code does, what the public and hidden objects are, and which visible diagnostics are built into the implementation.

The current design should therefore be read as a precise implementation note for a BA5-based two-dimensional CQA KEX prototype. The main open problems remain cryptanalytic: inversion of the hidden extractor, action-equivalent key recovery, transcript-level shared-secret recovery, and the study of image geometry under the visible shell.

The larger motivation remains the same as in the surrounding CQA project: to study whether HHS-like commutative actions can be realized in a much lighter arithmetic substrate than isogeny-based systems, while still preserving a meaningful action-based key-exchange semantics.[Cou06, CLM$^+$18, DF17]

10 How to run the script

The script is designed to be usable as a small standalone command-line tool. A minimal self-test can be run as follows:

```
python3 cqa_2d_kex.py --self-test
```

To print the full self-test report, use

```
python3 cqa_2d_kex.py --self-test --self-test-verbose
```

A typical demonstration run with explicit toy parameters is

```
python3 cqa_2d_kex.py \
--base-w 32 \
--tweak 0 \
--delta 2 \
--degree 3 \
--arity 5 \
--rounds 2 \
--scalar-rounds 1 \
--family-seed 12345 \
--u0 1 --u1 2 \
--g1 5 --g2 7 \
--seedA 111 --seedB 222 \
--out-len 32
```

To inspect the detailed BA5 branch traces, add

```
--trace-branches
```

to the demo command. The main command-line parameters are:

- `-base-w`, `-tweak`, `-delta`: dyadic-width and 2-adic family parameters,
- `-degree`, `-arity`: public family shape (with BA5 requiring `arity = 5`),
- `-rounds`: visible operator-round depth,
- `-scalar-rounds`: BA5 branch-iteration depth ρ,
- `-family-seed`: public family-generation seed,
- `-u0`, `-u1`: public branch seeds,
- `-g1`, `-g2`: public shell vector coordinates,
- `-seedA`, `-seedB`: deterministic toy seeds for the two secret tuples,
- `-out-len`: derived session-key length in bytes.

The script also enforces a few basic input conditions: `base-w` must be even, `degree` must be positive odd, and the BA5 design requires `arity = 5`. These checks are implementation guards rather than mathematical security statements.

11 Acknowledgements and Notes

This research has made intensive use of "GPT-5.2 Think deeper" and "GPT-5.4 Think deeper" as part of the tools NTNU offers to its members via a subscription to Microsoft 365 Copilot. Occasionally, I have also consulted Grok 4.2.

I would also like to thank NTNU "Beyond 5G Lab" for allowing me to use their computing facilities when testing the designs.

References

[CLM+18] Wouter Castryck, Tanja Lange, Chloe Martindale, Lorenz Panny, and Joost Renes. Csidh: An efficient post-quantum commutative group action. In *Advances in Cryptology – ASIACRYPT 2018*, volume 11274 of *Lecture Notes in Computer Science*, pages 395–427. Springer, 2018. Also available as IACR ePrint 2018/383.

[Cou06] Jean-Marc Couveignes. Hard homogeneous spaces. Technical Report 2006/291, IACR Cryptology ePrint Archive, 2006.

[DF17] Luca De Feo. Mathematics of isogeny based cryptography. *CoRR*, abs/1711.04062, 2017.

[Gli26] Danilo Gligoroski. Commutative-quasigroup-actions (cqa) proof-of-concept. `https://github.com/danilo-gligoroski/Commutative-Quasigroup-Actions`, 2026. GitHub repository, accessed 2026-03-12.

[MŠG10] S. Markovski, Z. Šunić, and D. Gligoroski. Polynomial functions on the units of $\mathbb{Z}_{2^n}$. *Quasigroups and Related Systems*, 23(1):59–82, 2010.